

Технический аудит — Next YOU / AIprofnavigator

Проведён 20 августа 2026 по запросу «не получается получить стабильный результат». Объект: `github.com/ivanvasiluk-maker/AIprofnavigator`, коммит `e202318` (main, 20.08.2026). Метод: чтение кода + локальный прогон тестов на Python 3.12 + разбор их собственной телеметрии. Живых вызовов OpenAI не делал — ключа нет, см. § 8 «Границы проверки». Дополнено 20.08 вечером: запись созвона команды, полный лог диалога с ботом (81 сообщение) и сам выданный HTML-отчёт — § 2. Там же одна поправка к первой версии.

1. Главный вывод

Нестабильность не в модели. Она заложена в архитектуре.

Один и тот же пользователь с одним и тем же рассказом может получить **один из шести разных результатов**, и какой именно — решает не его профиль, а скорость ответа OpenAI в конкретную минуту. При этом система нигде не падает честно: на каждом уровне стоит тихая подмена результата заглушкой, и ни пользователь, ни команда не видят, какой из шести путей сработал.

Три вещи, которые я считаю причиной жалобы, в порядке вклада:

1. **Таймауты вложены наоборот.** Внешний бюджет — 40 секунд, внутренний — 55. Полный качественный путь (генерация + починка) требует по их же замерам 1,5–3,5 минуты. То есть в проде он физически не успевает почти никогда, и пользователь получает детерминированную заглушку. Когда OpenAI отвечает быстро — получает настоящий отчёт. Это и есть «то работает, то нет».
2. **Последний рубеж сам падает.** Функция, которая должна срабатывать всегда, когда модель подвела, на реальном профиле (Краков, специалист по качеству) бросает исключение. Вызов не обёрнут в try. Результат для пользователя: бот молча замолкает на фразе «Собираю заключение».
3. **CI деплоит, но не тестирует.** В репозитории 384 теста и ноль из них запускается перед выкатом. Единственный workflow — `deploy-railway.yml`, он только деплоит. На текущем main три теста красные. Они уехали в прод.

Пункт 1 подтверждён прямым измерением, а не расчётом: в логе живого прогона между «Собираю заключение» и выдачей результата прошло **ровно 40 секунд** — в точности значение таймаута. Подробно — § 2.1.

Четвёртое добавилось после разбора лога и к таймаутам отношения не имеет: **каждое второе нажатие кнопки теряется, и в ответы пользователя записывается слово «Готово»**. Дальше система честно строит заключение по мусорным данным. Подробно — § 2.4.

Отдельно: **измерить стабильность сейчас нечем**. В аналитике не сохраняется, какой из путей выдал отчёт (`recovered_by` пишется в метаданные, но не в событие `report_generated`). Пока этого числа нет, любое «стало стабильнее» — ощущение, а не факт.

2. Живой прогон 20 августа: доказательства

Аудит кода делался по репозиторию и независимо. Потом появились три вещи, которых у меня не было: запись созвона команды, полный лог диалога Ивана Веденина с ботом (81 сообщение, 15:24–

15:41) и сам HTML-отчёт, который бот выдал. Ниже — что из этого подтвердилось, что уточнилось и что нашлось нового.

2.1. Главное: таймаут в 40 секунд не рассчитан, а измерен

В § 5.1 я оценивал время генерации расчётом. Лог диалога закрывает вопрос точно:

15:37:26 бот: «Собираю заключение»

15:38:06 бот: «Профессиональное ядро не уточнено ... Основной маршрут: не уточнено»

Ровно 40 секунд. Ровно столько, сколько стоит в `CAREER_ASSESSMENT_TIMEOUT_SECONDS` (`config.py:15`). Это не совпадение и не «модель так решила»: сработал внешний таймаут, генерация была прервана, и пользователю ушла детерминированная заглушка.

Диагноз из § 5.1 подтверждён на живых данных, а не выведен из расчёта.

2.2. Как выглядит заглушка: 40 раз «не уточнено» в одном документе

HTML-отчёт, который получил Иван, — структурно полный документ из 14 разделов. И почти пустой по содержанию. Дословно:

> **2. Профессиональная идентичность** — не уточнено > **5. Рекомендуемый маршрут** — не уточнено. «Маршрут "не уточнено" использует > подтверждённую основу "не уточнено"» > **6. Альтернативные маршруты** — «Маршрут пока не подтверждён» > **13. Первые шаги** — «Попросите одного специалиста по маршруту не уточнено проверить > список задач»

Строка `не уточнено` встречается в отчёте больше сорока раз, включая заголовки разделов, таблицу сравнения маршрутов и все три сценария развития.

И при этом отчёт заявляет о себе: «Уровень уверенности: средний». Это опаснее самой пустоты: человек, который дочитает до конца, увидит, что система в себе уверена.

2.3. Данные резюме БЫЛИ. Ядро всё равно не собралось

Это меняет диагноз по сравнению с первой версией аудита. Иван загрузил PDF-профиль из LinkedIn, и бот его разобрал: в 15:30:28 он перечислил 12 должностей за 20 лет. В финальный отчёт эти данные доехали частично — там есть образование («Master's degree, Computer Software Engineering, БГУИР, 1998–2004») и шесть навыков («Системное администрирование», «Управление проектами», «Продуктовый менеджмент», «Стратегическая фасилитация», «Системное мышление», «Дизайн хактонов»).

То есть системе было из чего собрать профессиональное ядро. Она написала «не уточнено».

Дело не в том, что пользователь мало рассказал. Дело в том, что сборка была прервана на 40-й секунде, а запасной механизм умеет переносить факты, но не умеет делать вывод.

Отдельно виден дефект разбора резюме: в 15:30:28 бот выдал должности и периоды **двумя несвязанными списками** — сначала все 12 названий подряд, потом все 12 диапазонов дат подряд. Кто когда работал — восстановить невозможно. Стаж, последовательность и seniority из такого разбора не выводятся в принципе.

2.4. Каждое второе нажатие кнопки теряется

В логе это видно как система, а не как случайность. Три места подряд:

15:33:26 бот: «Отмечено пунктов: 1 ... можно нажать "  Готово"»
15:33:28 Иван: «  Готово» → бот молчит 65 секунд
15:34:33 Иван: «  Готово» (повтор) → «Кажется, вы выбрали вариант из предыдущего вопроса.
Вы хотите сказать: ответ по текущему вопросу:  Готово?»
15:34:49 Иван: «  Да, именно так» → бот молчит
15:35:07 Иван: «  Да, именно так» → пошёл дальше

И ровно то же самое ещё раз в 15:35:28 → 15:35:35.

Механика читается так: **первое нажатие бот принимает и внутренне переводит диалог на следующий вопрос, но не отправляет ответ.** Пользователь видит тишину и жмёт ещё раз — и это второе нажатие приходит уже в следующий вопрос, где опознаётся как «вариант из предыдущего». Пользователь подтверждает, и в ответы записывается слово «  Готово» как содержательный ответ на вопрос.

Куда это попадает: `handlers/career.py:8136` и `:8550` кладут сырой текст сообщения в `qa_answers`, а `qa_answers` — основа профиля, из которого строится всё заключение.

Почему сообщение не отправляется — по логу не видно, и это отдельная причина убрать глухие `except Exception` (§ 5.7): если отправка падает, сейчас об этом не узнает никто.

Того же класса в этом же логе:

- на два ответа кнопкой («**Полностью самостоятельно**», «**Больше 2 лет**») бот ответил «Можно выбрать кнопку, не обязательно формулировать самому» — то есть не распознал собственную кнопку;
- ветка «**Не знаю, с чего начать**» (15:27:04) привела к «Пока данных мало» и не собрала историю вообще: рассказа у системы не было ни одного слова;
- страну **не спросили ни разу**, при этом в отчёте стоит «Страна проживания: не указана» и рядом «Валюта: EUR» — подставленная по умолчанию.

2.5. Поправка к первой версии аудита

В первой версии я написал, что «Боюсь отказов» приписано пользователю, который этого не говорил. **Это неверно, снимаю.** По логу Иван сам выбрал «Боюсь отказов» в 15:36:16 из списка внутренних барьеров.

Что остаётся верным и что действительно является дефектом:

1. В телеметрии команды за 12 августа событие `report_guardrail_failed` про «страх отказа как неподтверждённый факт» стоит за одну миллисекунду до `report_generated`. Там нарушение зафиксировано и отчёт не остановлен. Это отдельный случай и он в силе.
2. В отчёте Ивана психологический барьер попал в раздел «**8. Условия, при которых переход будет устойчивым**» — и обе строки раздела оказались одинаковой заглушкой: «Боюсь отказов: Проверяется только для формата, графика и задач конкретного маршрута.» «Деньги: Проверяется только для формата, графика и задач конкретного маршрута.» Шаблон подставлен без смысла, и это противоречит собственному правилу проекта «психологическое состояние меняет размер шага, а не маршрут».

2.6. Что сказала команда на созвоне и где это в коде

Что прозвучало на созвоне	Где это в коде
Оксана: «Сначала три минуты он висел. Потом следующий вопрос шесть минут висел, а потом вообще завис»	§ 5.1 (таймауты) и § 5.3 (незащищённое падение)
Василюк: «Ему не понравился город — и всё, отчёт просто завис»	§ 5.3. Плюс падающий тест по нормализации страны и города (<code>test_poland_normalized_routes</code>)
Оксана: «Один раз нормально сработает, другой раз вот такая херня»	§ 5.1: результат зависит от того, успела ли генерация в 40 секунд
Василюк: «Мне важно иметь устойчивые результаты. Если они просто устойчивые, меня это устроит»	§ 5.7 и задание 6 пакета: пока нет числа, устойчивость не проверяется
Василюк: «Локальный пропускаю и сразу иду на боевого»	§ 5.4: выкат без прогона тестов

2.7. Продуктовое, отдельно от кода

Прозвучало от человека, впервые прошедшего продукт целиком:

- **Путь долгий.** По логу: `/start` в 15:24:50, заключение в 15:38:06 — **13 минут**, и это в режиме «⚡ Быстро». Иван вслух: «очень долго, нудно, что-то надо тыкать, я уже устал».
- **Первые шаги отпугивают ценником времени:** «Уточнить приоритеты · 20 мин», «Проверить рынок · 45 мин». Иван: «Не уверен, что есть люди, готовые 45 минут тыкать в бота кнопочки».
- **Таблица сравнения маршрутов в HTML плохо отображается.**
- **Заключение обрывается там, где начинается польза.** Аналогия Ивана: «Вы сходили сдали анализы, вам выдали листок с цифрами, без специалиста вы их не поймёте». Напрашивается следующий шаг: предложить собрать резюме под выбранный маршрут, дать готовый промпт.

2.8. Поправка к § 4 (сверка с целями)

На созвоне Василюк сказал про треки и ежедневные задания: «У нас всё это реализовано, просто я сейчас целиком сосредоточился на заключении».

Проверил: в репозитории одна ветка, `main`. Слов «КПТ», «ДБТ», «АСТ», «балл», «points», «подписка» в коде нет вообще, а «План на 30 дней» существует как **раздел отчёта** в `utils/reporting.py`, то есть в старом контуре, выключенном флагом `LEGACY_CAREER_REPORT_ENABLED`.

Таблица в § 4 верна для того, что лежит в репозитории. Если работающая версия треков живёт где-то ещё — скажите, поправлю: это меняет оценку готовности продукта, но не меняет ничего в причинах нестабильности.

3. Что проект делает на самом деле

Telegram-бот на aiogram 3. Один сценарий:

/start → выбор темпа → рассказ (текст или голос) → подтверждение
→ (опционально CV: PDF/DOCX) → 5–12 уточняющих вопросов
→ сборка CareerAssessment одним вызовом модели
→ карта в Telegram + HTML-документ файлом (+ PDF)
→ пост-действия: выбор маршрута, разбор барьеров, ключевые слова, «к специалисту»

Размеры:

Что	Число
Прод-код (Python, без тестов)	28 063 строк
<code>handlers/career.py</code> — один файл	10 489 строк, 284 функции
Тесты	11 082 строк, 384 теста
Документация в репо	4 584 строк
Коммитов	76 (19 через PR, ветки <code>codex-*</code>)
Средний коммит за последние 12	+324 строки
Вызовов модели в коде	1 место (<code>openai_client.py:967</code>)
Модель	<code>gpt-4o-mini</code> , <code>temperature=0.2</code> , без <code>seed</code> , без <code>max_tokens</code>

Хостинг — Railway, деплой автоматом с каждого пуша в main.

Полезное, что стоит сохранить при любой переделке: разделение слоёв (`canonical_profile` → `evidence` → `routes`), валидатор с кодами ошибок (`RAW_USER_SUMMARY_AS_TITLE` , `GENERIC_ROUTE_TITLE` и т. п.), пакет эталонных профилей `tests/career_profiles/` и полный трейс генерации (`reports/career_assessment_profile_10_e2e_trace.json`). Это грамотные вещи, их мало у кого есть на этой стадии. Проблема не в том, что сделано плохо — проблема в том, что поверх этого не построен рабочий контур доставки.

4. Сверка с заявленными целями

Источник целей — `README.md` репозитория (раздел «Дополнения к техническому заданию») плюс досье проекта в трекинге.

Заявлено	В коде	Статус
Диагностика по 4 направлениям (профиль, психоэмоц., соц. поддержка, соц. интеграция)	Профиль — глубоко. Остальные три — как выбор кнопками, подмешиваемый в промпт текстом	частично
3–5 карьерных направлений	Есть, <code>routes.primary_routes</code> + альтернативы	да
Персональная карта развития	Есть, HTML + карта в Telegram	да
5–7 действий на неделю	Есть <code>first_steps</code> (3–5)	частично
Трек 2: психологическая устойчивость (КПТ, ДБТ, АСТ)	Слов «КПТ», «ДБТ», «АСТ» в коде нет ни одного	нет
Трек 3: социальная интеграция как отдельный трек	Только как фактор внутри карьерного вывода	нет
Ежедневное сопровождение (задания, отметка «получилось/нет»)	Есть <code>STEP_TRACKING</code> на один выбранный шаг. Ежедневного цикла нет	зачаток
Еженедельный чекпоинт	Упоминание в одном промпте, механики нет	нет
Геймификация, баллы, конвертация в скидку	Слов «балл», «points», «подписка», «скидка» в коде нет	нет
Монетизация 3 уровня	Уровень 2 — ссылка на специалиста, уровень 3 — ссылка на группу. Оплаты нет	зачаток
Постоянная БД	SQLite заведена, но на эфемерном диске (см. § 5.6)	частично

Что это значит продуктово. Заявлен продукт-сопровождение (диагностика → три трека → ежедневный цикл → удержание → подписка). Построен продукт-разовый-отчёт. Это не претензия: на текущей стадии сузиться правильно. Но их собственный аудит `mvp3_tz_audit_2026-06-29.md` фиксировал те же разрывы два месяца назад, и за это время закрывались не они, а качество отчёта — 27 патчей подряд по одному и тому же узлу.

5. Почему результат нестабилен

5.1. Таймауты вложены наоборот — главный подозреваемый

Три таймаута на одном пути:

Уровень	Значение	Где
Внешний бюджет всей сборки	40 с	<code>config.py:15</code> → <code>handlers/career.py:7349</code>
HTTP-клиент OpenAI	45 с, <code>max_retries=1</code>	<code>openai_client.py:957</code>
Внутренний на один вызов	55 с	<code>openai_client.py:1004</code>

Внешний меньше внутреннего. Значит внутренние таймауты и вся логика починки **никогда не доживают до срабатывания** — внешний убивает их первым.

Насколько это критично, видно из их же трейса (`reports/career_assessment_profile_10_e2e_trace.json`):

- вход модели: ~1 840 токенов;
- выход модели: 9 319 знаков ≈ 4 200 токенов на один CareerAssessment;
- в этом самом трейсе первая генерация **не прошла валидацию** (`RAW_USER_SUMMARY_AS_TITLE` , `INVALID_SENIORITY`), понадобился второй вызов — `repair` . То есть починка это норма, а не исключение.

Два вызова по 4 200 токенов выхода на `gpt-4o-mini` — это порядка полутора-трёх минут. Бюджет — 40 секунд. Дальше срабатывает `except Exception (handlers/career.py:7333)`, и пользователь получает детерминированную заглушку вместо отчёта.

Отсюда прямое следствие: чем медленнее OpenAI в конкретную минуту, тем хуже отчёт. Один и тот же профиль в 10:00 и в 10:05 даёт разные результаты. Ровно жалоба команды.

5.2. Шесть источников результата, и они неразличимы

Один и тот же пользователь может получить любое из:

#	Что	Где
1	Отчёт с первой генерации	<code>openai_client.py:1268</code>
2	Отчёт после починки моделью	<code>openai_client.py:1331</code>
3	Детерминированная сборка (валидация не прошла)	<code>openai_client.py:1385</code>
4	Детерминированная сборка (таймаут/ошибка)	<code>handlers/career.py:7356</code>
5	Детерминированная сборка «только из источников», без story и resume	<code>handlers/career.py:7387</code>
6	Захардкоженный текст «Предварительное заключение»	<code>handlers/career.py:409</code> и <code>:475</code>

Плюс седьмой слой в `_run_json (openai_client.py:1009)`: при **любом** исключении возвращается заранее написанный словарь-заглушка — без логов, без метки.

Пользователю все шесть приходят одинаково оформленными. Команде — тоже: событие `report_generated` в телеметрии несёт только `has_income_signal`, поля `recovered_by` в нём нет. Поэтому нестабильность и невозможно поймать: она не оставляет следа.

5.3. Последний рубеж сам падает — и падение не поймано

`build_deterministic_assessment` — та самая функция, которая должна работать всегда. Она заканчивается вызовом `.require_valid() (services/career_assessment.py:1896)`, который бросает `ValueError`.

Проверено прогоном: на реальном профиле «Краков, специалист по качеству, 7200 PLN» (`tests/test_poland_normalized_routes.py`) она падает:

```
ValueError: Invalid CareerAssessment: DUPLICATE_ROUTE_ANALYSIS at routes:
routes must have distinct rationale, gap, risk, test and entry path
```

Дальше: `_build_and_send_report` вызывается в трёх местах — `handlers/career.py:7681`, `:8860`, `:9901` — ни одно не обернуто в `try`.

Полная цепочка отказа: модель не уложилась в 40 с → ловим исключение → зовём детерминированную сборку → она бросает `ValueError` → он никем не пойман → `aiogram` проглатывает исключение хендлера → **пользователь остаётся в состоянии «Собираю заключение» навсегда**. Ни ответа, ни ошибки, ни возможности повторить.

Это, по-моему, и есть то, что команда видит как «бот завис».

5.4. CI выкатывает непроверенный код

`.github/workflows/deploy-railway.yml` — единственный workflow в репозитории: `push в main` → `railway up`. Шага тестов нет.

Мой прогон на чистом окружении (Python 3.12, HEAD e202318):

3 failed, 381 passed in 60.03s

Красные:

- `test_poland_normalized_routes.py::test_skill_clusters_produce_distinct_ranked_transition_routes` — см. § 5.3;
- `test_render_ru_be.py::test_missing_route_context_does_not_block_report_generation` — сборка отчёта не запускается там, где должна («Expected mock to have been awaited once. Awaited 0 times»);
- `test_baseline_lock.py::test_locked_file_hashes_match_current_workspace` — их собственный механизм защиты от незамеченных правок разошёлся с кодом.

Второй тест особенно неприятен: он проверяет ровно ту гарантию, что нехватка данных о маршруте **не должна** блокировать выдачу отчёта. Он красный. Значит гарантии нет.

Третий показывает, что механизм контроля изменений, который они специально построили (`baseline_lock`, `change_proposals`, governance-гейты), сейчас не работает — в их же отчёте `reports/CODEX_CAREER_ASSESSMENT_REPAIR_REPORT.md` написано, что гейт отклоняет их собственную заявку на изменение.

5.5. Один вызов модели делает всю работу

Единственный вызов (`openai_client.py:967`) получает:

- промпт `CareerAssessment` — ~7 000 знаков, из них около **50 правил, в основном запретов** («не предлагай», «запрещено», «не создавай», «не психологизируй»...);
- JSON-схему на 7 150 знаков, **1 163 листовых поля** (с учётом массивов);
- и должен за один проход определить ядро профессии, seniority, доказательства, ограничения, 8–12 карьерных моделей, из них 3–5 маршрутов, анализ рынка по каждому, три сценария дохода по каждому, вопросы, выводы и 3–5 первых шагов.

`gpt-4o-mini` при таком объёме правил держит их выборочно и по-разному от прогона к прогону. Отсюда классическое «то учитывает отказ пользователя от функции, то нет».

Легаси-ветка (`LEGACY_CAREER_REPORT_ENABLED`) устроена ещё тяжелее: `FINAL_REPORT_PROMPT` — **32 455 знаков**, схема — 18 154 знака. По умолчанию выключена, но код живой и тестируется, то есть тянет внимание на себя.

Мелочи того же класса: `temperature=0.2` без `seed` (детерминизма не даёт), `max_tokens` не задан (нет защиты от обрыва длинной генерации).

5.6. Состояние живёт в памяти процесса, а деплои идут по несколько в день

- `bot.py:150` — `Dispatcher()` без хранилища, то есть `MemoryStorage`: всё интервью держится в памяти процесса.
- `handlers/career.py:310–311` — глобальные словари `SESSION_FSM_CACHE` и `SESSION_MESSAGE_CACHE`, в них складываются объекты `Message`. Они не чистятся.
- Чекпоинт в SQLite (`SessionCheckpointMiddleware`) пишется в `reports/app_data.sqlite3`. В `railway.json` тома нет; если том не подключён в дашборде Railway, файл живёт до первого передеплоя.

За один только 20 августа в main ушло 6 коммитов. **Каждый деплой обрывает все активные интервью.** Если тестировщик проходил бота в этот момент — он увидит именно «результат нестабилен», хотя код тут ни при чём.

Плюс `delete_webhook(drop_pending_updates=False)` при старте: накопленные апдейты переигрываются после рестарта, а защита от дублей (`DedupMiddleware`) хранит увиденные `update_id` в памяти и после рестарта пуста. То есть после каждого деплоя часть сообщений может обработаться повторно.

5.7. Что мешает диагностике

- **excerpt Exception без логирования** — 41 штука в `handlers/career.py`, 24 в `utils/persistence.py`. Ключевой: `openai_client.py:1009` — глотает ошибку API, лимит, невалидный JSON и отказ модели одинаково, возвращая заглушку.
- **Телеметрии почти нет:** в `reports/behavior_events.jsonl` **52 события, 4 пользователя, все за один день 12 августа.** Из 11 начатых сессий до `report_generated` дошла 1. Два `report_guardrail_failed` и один `report_draft_empty_blocked`. Это, кстати, единственные реальные цифры о продукте, которые у команды есть.
- Событие `interview_ready_with_uncertainty` пишется дважды (напрямую и как алиас от `interview_ready_early`, `handlers/career.py:1844` и `:1931`) — в данных 12 августа это видно. То есть даже эти 52 события нельзя считать наивно.
- **Тесты пишут в боевой путь:** после моего прогона в `reports/` появился `app_data.sqlite3` — сюита ходит в ту же базу, что и прод. В git он не попадает (`.gitignore`), но изоляции нет.

5.8. Структурная хрупкость (не баг сегодня, но производит баги завтра)

- `handlers/career.py` — 10 489 строк, 284 функции, 149 вызовов `update_data`, 262 разных ключа состояния. Ни один человек не держит это в голове целиком.
- В `states.py` рядом с 33 каноническими состояниями объявлены **15 алиасов на те же объекты** (`waiting_for_answers = INTERVIEW`, `waiting_for_post_result_action = FINAL_READY` и т. д.). Хендлеры регистрируются под обоими именами вперемешку: на состоянии `FINAL_READY / waiting_for_post_result_action` их 23 штуки. Кое-где один и тот же хендлер навешан дважды на одно и то же состояние (`handlers/career.py:9590-9591`, `:9618-9619`) — авторы не знали, что это одно состояние. Порядок срабатывания определяется порядком строк в файле: новый хендлер, добавленный не туда, молча меняет поведение старого.
- 14 разных маркеров `PATCH-NN` в коде — археология правок вместо структуры.

6. Что чинить

Сегодня (полдня, максимальный эффект на «стабильность»)

1. **Развести таймауты.** Внешний бюджет должен быть больше суммы внутренних, а не меньше. Минимально: внешний 180 с, HTTP 60 с, внутренний 60 с. Это одна строка в `config.py` и две в `openai_client.py`. Без этого всё остальное бессмысленно — качественный путь в проде просто не запускается.
2. **Обернуть `build_deterministic_assessment` в try** во всех трёх местах и в самом плохом случае отдавать пользователю честное «не смог собрать, нажмите «Повторить»» вместо тишины.
3. **Добавить шаг тестов в CI перед деплоем** — десять строк в `deploy-railway.yml`, `pytest tests -q` перед `railway up`. И починить три красных теста.

4. Начать писать `recovered_by` в событие `report_generated`. До этого «стабильность» не измеряется, а обсуждается.

На неделю (лечение причины)

5. Разбить один мега-вызов на 3–4 маленьких. Профиль → маршруты → рынок → шаги. Каждый со своей маленькой схемой, выход ≤ 800 токенов, каждый валидируется и перезапрашивается отдельно. Это главный ход: короткая генерация и быстрее, и стабильнее на порядок, и падает точно, а не целиком.
6. Перенести запреты из промпта в код. Правило «не предлагать административную работу без доказательств» — это проверка на выходе, а не строка в промпте. У них уже есть валидатор с кодами ошибок, туда и добавлять.
7. Вынести состояние из памяти процесса — Redis на Railway (есть в один клик) либо том под SQLite. Тогда деплой перестает ронять живые сессии.
8. Перед выкатом — предупреждение тестировщикам или окно деплоя. Пока состояние в памяти, выкат во время теста гарантированно даёт «нестабильный результат».

На месяц

9. Завести метрику стабильности. У них уже есть почти всё нужное: `tests/career_profiles/` с 10 эталонными профилями. Не хватает одного — прогонять каждый профиль **N раз** и считать долю расхождений по ключевым полям (профессиональное ядро, основной маршрут, `seniority`). Сейчас прогон один, а разброс между прогонами и есть предмет жалобы. Число вида «7 из 10 профилей дают один и тот же основной маршрут в 3 прогонах из 3» — вот что нужно на доске.
10. Разобрать `handlers/career.py` по слоям (диалог / сборка / доставка). Не ради красоты: сейчас цена любой правки — риск сломать соседнее, и именно поэтому цикл патчей не кончается.
11. Удалить легаси-ветку отчёта целиком, если она выключена.

Организационное (это к трекингу, не к коду)

Двадцать семь патчей подряд по одному узлу — это признак работы над экземплярами вместо класса. Ни один из перечисленных выше дефектов не чинится патчем к промпту: три из них вообще в конфигурации и в CI. Рекомендую следующий круг посвятить не качеству отчёта, а контуру доставки — таймауты, тесты в CI, метрика стабильности. Иначе следующие 27 патчей уйдут туда же.

7. Что в проекте сделано хорошо

Чтобы не сложилось однобокой картины — это не слабый репозиторий:

- Разделение `canonical_profile` / `evidence` / `routes` с обязательными ссылками маршрута на факты — правильная защита от выдумывания биографии.
- Валидатор с машиночитаемыми кодами ошибок и структурой `field_path` — редко встречается на этой стадии.
- Полный трейс генерации со снимком сырого вывода модели до починки — именно то, что нужно для разбора инцидентов. Его не хватает только в проде.
- Пакет эталонных профилей с ожидаемыми результатами и отдельными оценщиками (`evidence`, `seniority`, `route`, `forbidden_recommendation`) — это заготовка под метрику стабильности, до которой остался один шаг.

- Изоляция тестовых профилей от прода (`services/runtime_isolation.py`) — думали о правильных вещах.

Материал для устойчивого продукта есть. Не хватает не ума, а контура: тесты в CI, согласованные таймауты, честное падение и одно число, по которому видно, стало лучше или нет.

8. Границы проверки

Чего я не делал и что стоит проверить команде:

- **Живых вызовов OpenAI не было** — ключа у меня нет. Оценка времени генерации (1,5–3,5 минуты на два вызова) — расчёт от измеренного объёма вывода (4 200 токенов), а не замер. Проверяется за 10 минут: залогировать реальную длительность `_chat` в проде. Если она регулярно больше 40 секунд — § 5.1 подтверждён полностью.
- **Поведение на Railway** оценивал по коду и конфигам. Подключён ли том под SQLite — видно только в дашборде Railway.
- **PDF-движки** (playwright → xhtml2pdf → reportlab) не прогонял: три движка дают три разных вида документа, но на «стабильность результата» в смысле содержания не влияют.
- **Безопасность**: ключи в истории репозитория не искал вручную — прогнал поиск по всем коммитам на паттерны OpenAI и Telegram-токенов, **утечек не нашёл**. `.env` в `.gitignore` с самого начала.

Воспроизведение прогона тестов:

```
git clone https://github.com/ivanvasiluk-maker/AIprofnavigator && cd AIprofnavigator && python -m ve
```

Аудит для трекинга Next YOU. Парный к `NextYOU_анализ_сессия5.md`. Факты о команде и сроках — в `СОСТОЯНИЕ.md`, задачи — в `ЗАДАЧИ.md`.